TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 6: Máy vectơ hỗ trợ (SVM)

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Đường có lề rộng nhất **2**
2.1 Nhắc lại ý tưởng . . . 2
2.2 Đọc kết quả . . . 5
2.3 Dự đoán cho điểm mới . . . 5

3 Lề mềm và tham số C **6**
3.1 Đọc bảng thứ nhất . . . 7
3.2 Đọc bảng thứ hai . . . 8

4 Kernel và biên cong **8**
4.1 Nâng chiều bằng tay . . . 8
4.2 Để kernel làm việc đó . . . 8
4.3 Đọc kết quả . . . 10

5 Chọn C và gamma **11**
5.1 Ảnh hưởng của gamma . . . 12
5.2 Dò lưới . . . 12

6 Nhận dạng chữ số viết tay **13**
6.1 Đọc kết quả . . . 15
6.2 So với các mô hình khác . . . 15

7 Bài tập **15**

8 Phụ lục A, bảng tra nhanh **17**

9 Phụ lục B, lỗi thường gặp **18**

1. Mục tiêu bài thực hành

Hồi quy logistic vẽ một đường thẳng bằng cách dùng mọi điểm dữ liệu. KNN vẽ được biên cong nhưng phải giữ lại toàn bộ dữ liệu. Buổi này học SVM, mô hình chọn đường ranh giới cách xa cả hai lớp nhất, chỉ cần nhớ vài điểm nằm sát ranh giới, và nhờ kernel vẽ được cả biên cong.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Học một SVM tuyến tính, đọc ra đường ranh giới, độ rộng của lề và các vectơ hỗ trợ.
2. Giải thích tác dụng của tham số C qua số vectơ hỗ trợ và độ rộng lề.
3. Kiểm chứng rằng SVM phải chuẩn hoá khi các cột khác thang đo.
4. Thấy ý tưởng nâng chiều bằng tay, rồi dùng kernel RBF trên dữ liệu có biên cong.
5. Chọn cặp C và gamma bằng `GridSearchCV`.
6. Dùng SVM nhận dạng chữ số viết tay và so với các mô hình đã học.

Bốn bộ dữ liệu

Mục 2 tự sinh 24 điểm bằng hàm `make_blobs` của scikit-learn. Mục 3 dùng lại `data/sinh_vien.csv` và `data/sinh_vien_mo_rong.csv`. Mục 4 và mục 5 dùng lại `data/hai_vang_trang.csv`. Mục 6 dùng bộ 1797 ảnh chữ số viết tay có sẵn trong scikit-learn, nạp bằng `load_digits`, không cần tải gì thêm.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai06_svm
```

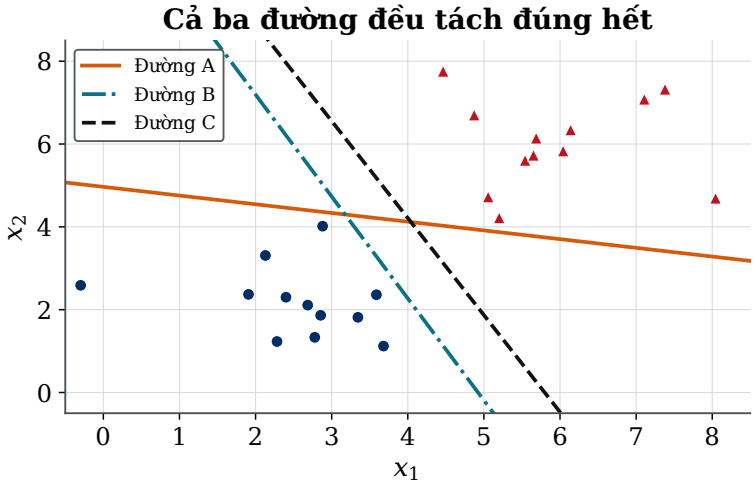
Bảng 1: *Năm tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`g1_le_rong_nhat.py`	Đường có lề rộng nhất và vectơ hỗ trợ
3	`g2_tham_so_c.py`	Tham số C và việc chuẩn hoá
4	`g3_kernel.py`	Nâng chiều bằng tay và kernel
5	`g4_chon_tham_so.py`	Gamma và dò lưới
6	`g5_chu_so.py`	Nhận dạng chữ số viết tay

2. Đường có lề rộng nhất

2.1. Nhắc lại ý tưởng

Khi hai lớp tách hẳn nhau, có vô số đường thẳng tách đúng hết. SVM chọn đường nằm giữa **con đường rộng nhất** có thể kẻ được giữa hai lớp.



Hình 1: *Ba đường A, B, C cùng tách đúng hết 24 điểm, nhưng đường C cách xa cả hai lớp nhất.*

Đường ranh giới và lề

Đường ranh giới là tập các điểm có $f(\boldsymbol{x}) = 0$, với

$$f(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + b$$

SVM đoán lớp 1 khi $f(\boldsymbol{x}) > 0$ và lớp 0 khi $f(\boldsymbol{x}) < 0$. Hai mép lề là hai đường $f(\boldsymbol{x}) = 1$ và $f(\boldsymbol{x}) = -1$, cách nhau một khoảng

$$\text{lề} = \frac{2}{\sqrt{w_1^2 + w_2^2}}$$

Những điểm nằm đúng trên mép lề gọi là **vectơ hỗ trợ**.

Tệp mã dưới đây học SVM tuyến tính trên 24 điểm, in ra w_1, w_2, b, độ rộng lề và các vectơ hỗ trợ. Tham số `C=1000` rất lớn nghĩa là không cho phép điểm nào nằm trong lề, mục 3 sẽ nói kỹ về tham số này.

```
# -*- coding: utf-8 -*-
"""Bước 1: tìm đường có lề rộng nhất và các vectơ hỗ trợ.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""

import numpy as np
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

# 24 điểm, hai cụm tách hẳn nhau
X, y = make_blobs(n_samples=24, centers=[[2, 2], [6, 6]],
                  cluster_std=0.9, random_state=0)
y = 1 - y            # lớp 1 là cụm dưới bên trái, giống hình trên slide

# C rất lớn nghĩa là không cho điểm nào nằm trong lề
```

```python
svm = SVC(kernel="linear", C=1000)
svm.fit(X, y)

w = svm.coef_[0]
b = svm.intercept_[0]
print(f"w1 = {w[0]:.3f}, w2 = {w[1]:.3f}, b = {b:.3f}")
do_dai_w = np.sqrt(w[0] ** 2 + w[1] ** 2)
print(f"Do rong cua le = 2 / {do_dai_w:.4f} = {2 / do_dai_w:.4f}")
print()

# Các vectơ hỗ trợ nằm đúng trên hai mép lề, nơi f(x) bằng 1 hoặc -1
print("So vecto ho tro:", len(svm.support_))
for i in svm.support_:
    f = svm.decision_function(X[i:i + 1])[0]
    print(f"  diem {i:2d}: x1 = {X[i, 0]:.2f}, x2 = {X[i, 1]:.2f},"
          f" lop {y[i]}, f(x) = {f:+.3f}")
print()

# Những điểm khác nằm ngoài lề, nên |f(x)| lớn hơn 1
f_tat_ca = svm.decision_function(X)
ngoai = np.setdiff1d(np.arange(len(X)), svm.support_)
nho_nhat = np.abs(f_tat_ca[ngoai]).min()
print(f"|f(x)| nho nhat cua cac diem khac: {nho_nhat:.3f}")
print()

# Học lại chỉ với các vectơ hỗ trợ
chi_sv = SVC(kernel="linear", C=1000)
chi_sv.fit(X[svm.support_], y[svm.support_])
w2 = chi_sv.coef_[0]
print("Hoc lai chi voi vecto ho tro:")
b2 = chi_sv.intercept_[0]
print(f"  w1 = {w2[0]:.3f}, w2 = {w2[1]:.3f}, b = {b2:.3f}")
print()

# Dự đoán cho hai điểm mới
moi = np.array([[3.0, 3.0], [5.0, 4.5]])
for diem, f, lop in zip(moi, svm.decision_function(moi),
                        svm.predict(moi)):
    print(f"Diem ({diem[0]}, {diem[1]}): f(x) = {f:+.3f}, doan lop {lop}")
```

```
python code/g1_le_rong_nhat.py
```

```
w1 = -0.831, w2 = -0.355, b = 4.820
Do rong cua le = 2 / 0.9039 = 2.2126

So vecto ho tro: 3
  diem 18: x1 = 5.20, x2 = 4.22, lop 0, f(x) = -1.000
  diem 10: x1 = 2.88, x2 = 4.02, lop 1, f(x) = +1.000
```

```
  diem 13: x1 = 3.59, x2 = 2.36, lop 1, f(x) = +1.000

|f(x)| nho nhat cua cac diem khac: 1.059

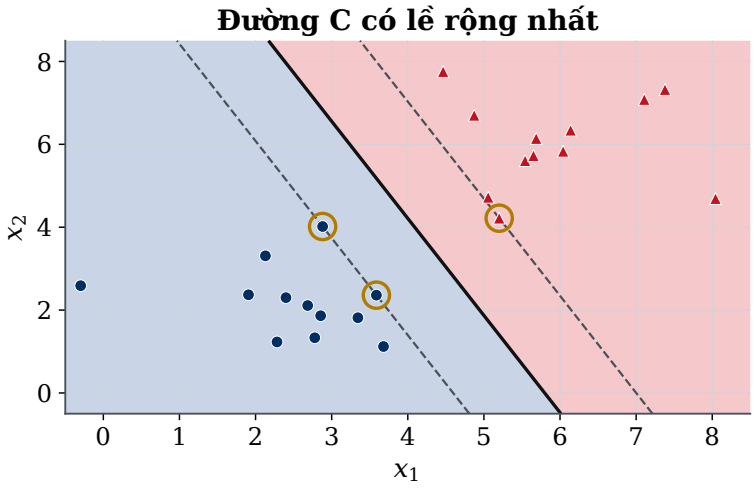
Hoc lai chi voi vecto ho tro:
  w1 = -0.831, w2 = -0.355, b = 4.821

Diem (3.0, 3.0): f(x) = +1.262, doan lop 1
Diem (5.0, 4.5): f(x) = -0.933, doan lop 0
```

2.2. Đọc kết quả

Với $w_1 = -0.831$ và $w_2 = -0.355$, độ dài của vectơ $\boldsymbol{w}$ là 0.9039, nên lề rộng $2/0.9039 \approx 2.21$.

Có đúng 3 vectơ hỗ trợ: một điểm của lớp 0 có $f(\boldsymbol{x}) = -1$, hai điểm của lớp 1 có $f(\boldsymbol{x}) = +1$. Chúng nằm đúng trên hai mép lề. Mọi điểm còn lại có $|f(\boldsymbol{x})|$ ít nhất là 1.059, tức là nằm ngoài lề.



Hình 2: *Đường ranh giới (nét liền), hai mép lề (nét đứt) và ba vectơ hỗ trợ (khoanh vàng).*

Chỉ vectơ hỗ trợ quyết định đường ranh giới

Học lại SVM chỉ với 3 vectơ hỗ trợ, ta được gần như đúng w_1, w_2 và b cũ. b lệch ở chữ số thập phân thứ ba chỉ là sai số của phép tính gần đúng bên trong thư viện. 21 điểm còn lại không góp phần nào vào đường ranh giới.

2.3. Dự đoán cho điểm mới

Điểm $(3, 3)$ có $f = +1.262$, lớn hơn 1, nên nằm ngoài lề về phía lớp 1. Điểm $(5, 4.5)$ có $f = -0.933$, âm nên được đoán lớp 0. Nhưng $|f| < 1$ cho biết điểm này nằm **bên trong lề**, gần ranh giới, nên đây là một dự đoán kém chắc chắn.

f(x) không phải xác suất

Giá trị $f(\boldsymbol{x})$ của SVM chỉ cho biết điểm nằm phía nào và cách ranh giới bao xa. Nó không nằm trong đoạn từ 0 tới 1 như xác suất của hồi quy logistic.

3. Lề mềm và tham số C

Dữ liệu thật hiếm khi tách sạch. Trong bộ sinh viên của bài 2, có bạn ôn nhiều mà vẫn rớt, có bạn ôn ít mà vẫn qua, nên không đường thẳng nào tách đúng hết. SVM khi đó dùng **lề mềm**: cho phép một số điểm nằm trong lề hoặc sai phía, mỗi điểm như vậy chịu một mức phạt.

Tham số C

C là mức phạt cho mỗi điểm vi phạm lề. C nhỏ thì phạt nhẹ, SVM ưu tiên lề rộng và chấp nhận nhiều điểm vi phạm. C lớn thì phạt nặng, SVM bóp lề hẹp lại để xếp đúng càng nhiều điểm càng tốt. Mọi điểm nằm trên mép lề, trong lề hoặc sai phía đều là vectơ hỗ trợ.

```python
# -*- coding: utf-8 -*-
"""Bước 2: lề mềm, tham số C, và vì sao SVM phải chuẩn hoá."""

import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

df = pd.read_csv("data/sinh_vien.csv")
X = df[["gio_on", "diem_giua_ky"]]
y = df["qua_mon"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

print("Tap hoc co", len(X_tr), "ban")
print("    C    so vecto HT   do rong le   tap hoc   tap kiem tra")
for C in [0.01, 0.1, 1, 10, 100]:
    mh = make_pipeline(StandardScaler(),
                       SVC(kernel="linear", C=C))
    mh.fit(X_tr, y_tr)
    svm = mh[-1]                    # bước cuối của pipeline là SVM
    le = 2 / np.linalg.norm(svm.coef_[0])
    a_hoc = accuracy_score(y_tr, mh.predict(X_tr))
    a_kt = accuracy_score(y_te, mh.predict(X_te))
    print(f"  {C:6}   {len(svm.support_):6d}       {le:6.2f}"
          f"       {a_hoc:.4f}    {a_kt:.4f}")
print()

# Bộ sinh viên mở rộng có thêm cột chi tiêu, tính bằng nghìn đồng
df3 = pd.read_csv("data/sinh_vien_mo_rong.csv")
X3 = df3[["gio_on", "diem_giua_ky", "chi_tieu"]]
y3 = df3["qua_mon"]
X3_tr, X3_te, y3_tr, y3_te = train_test_split(
    X3, y3, test_size=0.25, random_state=17, stratify=y3
```

```
)
tho = SVC().fit(X3_tr, y3_tr)
chuan = make_pipeline(StandardScaler(), SVC()).fit(X3_tr, y3_tr)
print("Ba cot, kernel RBF mac dinh")
print(f"  Doan bua ca lop qua : {y3_te.mean():.4f}")
print(f"  Khong chuan hoa     : "
      f"{accuracy_score(y3_te, tho.predict(X3_te)):.4f}")
print(f"  Co chuan hoa        : "
      f"{accuracy_score(y3_te, chuan.predict(X3_te)):.4f}")
```

```
python code/g2_tham_so_c.py
```

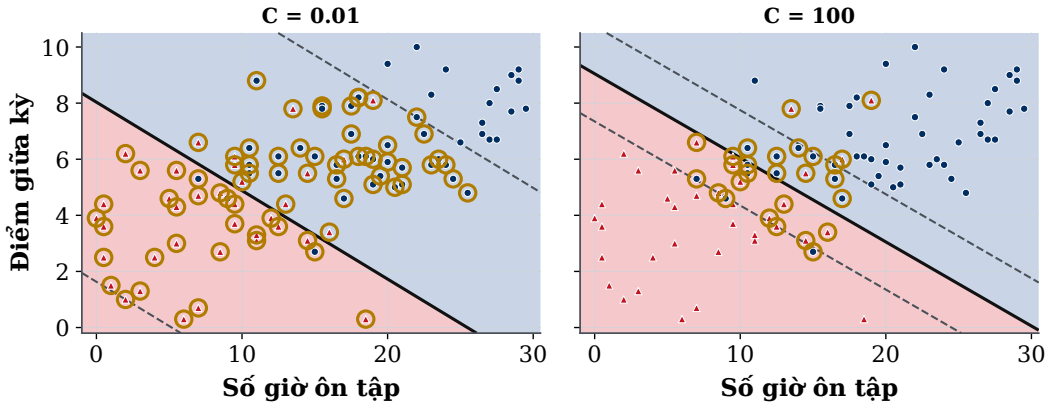
```
Tap hoc co 90 ban
    C    so vecto HT   do rong le   tap hoc   tap kiem tra
    0.01       74         3.84      0.8556    0.8667
     0.1       44         1.70      0.8889    0.9000
       1       31         1.29      0.8778    0.9000
      10       28         1.06      0.9000    0.9000
     100       27         1.05      0.9000    0.9000

Ba cot, kernel RBF mac dinh
  Doan bua ca lop qua : 0.6000
  Khong chuan hoa     : 0.6000
  Co chuan hoa        : 0.9000
```

3.1. Đọc bảng thứ nhất

Khi C tăng từ 0.01 lên 100:

- Số vectơ hỗ trợ giảm từ 74 xuống 27 trên 90 bạn của tập học.
- Độ rộng lề giảm từ 3.84 xuống 1.05, đo trên dữ liệu đã chuẩn hoá.
- Accuracy trên tập kiểm tra tăng từ 0.8667 lên 0.9000 rồi đứng yên.



Hình 3: *Cùng một tập học với $C = 0.01$ và $C = 100$. Các điểm khoanh vàng là vectơ hỗ trợ.*

Hình 3 cho thấy rõ sự khác biệt. Với $C = 0.01$, hai mép lề cách nhau rất xa, gần như mọi bạn nằm gần ranh giới đều thành vectơ hỗ trợ. Với $C = 100$, lề hẹp lại và chỉ những bạn thật sát ranh giới mới còn là vectơ hỗ trợ.

Ở đây C ảnh hưởng ít

Với ranh giới thẳng và chỉ hai cột, các giá trị C từ 0.1 trở lên cho cùng accuracy 0.9000, bằng hồi quy logistic của bài 2. C quan trọng hơn nhiều khi dùng kernel, như mục 5 sẽ cho thấy.

3.2. Đọc bảng thứ hai

Không chuẩn hoá, SVM chỉ đạt 0.6000, đúng bằng mức đoán bừa cả lớp qua. Có chuẩn hoá, SVM đạt 0.9000.

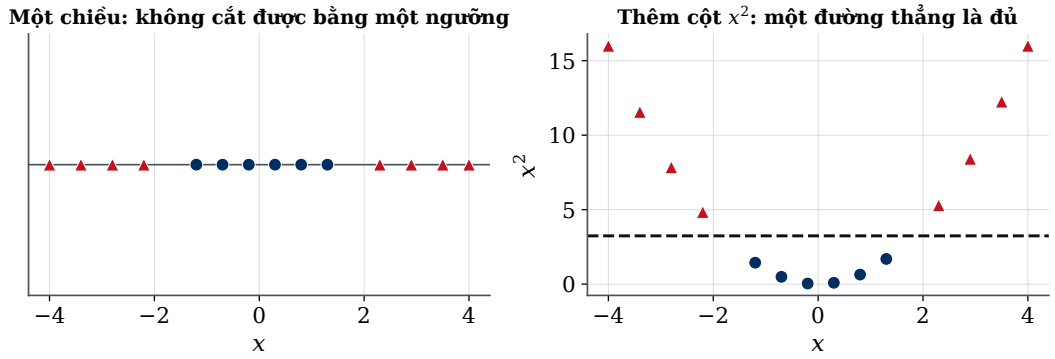
SVM gặp đúng cái bẫy của KNN

Lề là một khoảng cách, và kernel RBF cũng tính bằng khoảng cách. Cột chi tiêu tính bằng nghìn đồng có độ lệch chuẩn cỡ hàng nghìn, nên nuốt trọn khoảng cách, hai cột học tập gần như vô hình. Khi các cột khác thang đo, luôn dùng `make_pipeline(StandardScaler(), SVC())`.

4. Kernel và biên cong

4.1. Nâng chiều bằng tay

Xét 14 điểm trên một trục số, lớp 1 nằm giữa, lớp 0 nằm ở hai đầu. Không có ngưỡng nào trên trục x tách được, vì lớp 0 nằm ở cả hai phía. Nhưng nếu thêm cột x^2, lớp 1 có x^2 nhỏ còn lớp 0 có x^2 lớn, và một đường thẳng nằm ngang là đủ.



Hình 4: *Bên trái, dữ liệu một chiều không tách được bằng một ngưỡng. Bên phải, sau khi thêm cột x^2, một đường thẳng tách đúng hết.*

4.2. Để kernel làm việc đó

Với dữ liệu nhiều cột, số cột mới cần thêm tăng rất nhanh. May mắn là SVM không cần tới các cột mới, nó chỉ cần biết hai điểm giống nhau tới đâu. Hàm đo độ giống nhau đó gọi là **kernel**.

Kernel RBF

$$K(\boldsymbol{x}, \boldsymbol{z}) = e^{-\gamma \|\boldsymbol{x}-\boldsymbol{z}\|^2}$$

Hai điểm trùng nhau thì $K = 1$, càng xa nhau thì K càng gần 0. Đây là kernel mặc định của `SVC`, và γ là tham số `gamma`.

Tệp mã dưới đây làm cả hai việc: nâng chiều bằng tay cho 14 điểm, rồi so ba loại kernel trên bộ hai vầng trăng của bài 4.

```python
# -*- coding: utf-8 -*-
"""Bước 3: nâng chiều bằng tay, rồi để kernel làm việc đó."""

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC

# Phần 1: dữ liệu một chiều, lớp 1 nằm giữa, lớp 0 ở hai đầu
x = np.array([-4.0, -3.4, -2.8, -2.2, -1.2, -0.7, -0.2,
              0.3, 0.8, 1.3, 2.3, 2.9, 3.5, 4.0])
y = (np.abs(x) < 1.8).astype(int)

# Thử mọi ngưỡng: đoán lớp 1 khi x nhỏ hơn hoặc bằng ngưỡng
tot = max(((x <= t).astype(int) == y).mean() for t in x)
print(f"Mot nguong tren truc x, tot nhat dung {tot:.4f}")

# Thêm cột x bình phương rồi dùng SVM tuyến tính
X_moi = np.c_[x, x ** 2]
svm = SVC(kernel="linear", C=1000).fit(X_moi, y)
print(f"Them cot x^2, SVM tuyen tinh dung {svm.score(X_moi, y):.4f}")
print()

# Phần 2: hai vầng trăng, để kernel tự nâng chiều
df = pd.read_csv("data/hai_vang_trang.csv")
X = df[["x1", "x2"]]
y = df["nhan"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

mo_hinh = {
    "SVM kernel tuyen tinh": SVC(kernel="linear"),
    "SVM kernel da thuc bac 3": SVC(kernel="poly", degree=3),
    "SVM kernel RBF": SVC(kernel="rbf"),
    "Hoi quy logistic": LogisticRegression(),
    "KNN, k = 13": KNeighborsClassifier(n_neighbors=13),
}
print("Hai vang trang, accuracy tren tap kiem tra")
for ten, mh in mo_hinh.items():
    mh.fit(X_tr, y_tr)
    acc = accuracy_score(y_te, mh.predict(X_te))
    print(f"  {ten:26s} {acc:.4f}")
print()
```

```
48
49 rbf = mo_hinh["SVM kernel RBF"]
50 print("So vecto ho tro cua kernel RBF:", len(rbf.support_),
51       "tren", len(X_tr), "diem hoc")
```

```
python code/g3_kernel.py
```

```
Mot nguong tren truc x, tot nhat dung 0.7143
Them cot x^2, SVM tuyen tinh dung 1.0000

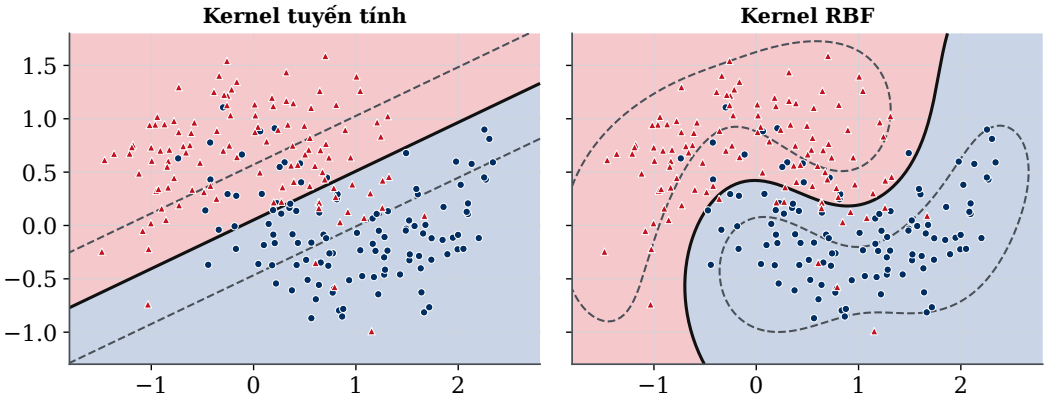
Hai vang trang, accuracy tren tap kiem tra
  SVM kernel tuyen tinh      0.8667
  SVM kernel da thuc bac 3   0.8667
  SVM kernel RBF             0.9200
  Hoi quy logistic           0.8533
  KNN, k = 13                0.9200

So vecto ho tro cua kernel RBF: 89 tren 225 diem hoc
```

4.3. Đọc kết quả

Trên trục số, ngưỡng tốt nhất chỉ đúng 0.7143, tức 10 trên 14 điểm. Thêm cột x^2, SVM tuyến tính đúng cả 14 điểm.

Trên hai vầng trăng, kernel tuyến tính và kernel đa thức bậc 3 cùng được 0.8667. Kernel RBF được 0.9200, ngang KNN với $k = 13$ của bài 4 và hơn hẳn hồi quy logistic.



Hình 5: *Biên quyết định của SVM kernel tuyến tính và kernel RBF trên bộ hai vầng trăng. Nét đứt là hai mép lề.*

SVM chỉ nhớ các vectơ hỗ trợ

SVM kernel RBF có 89 vectơ hỗ trợ trên 225 điểm học. Khi dự đoán, mỗi vectơ hỗ trợ bỏ một phiếu cho lớp của mình, phiếu nặng hay nhẹ tuỳ nó giống điểm mới tới đâu. KNN thì phải giữ cả 225 điểm và hỏi 13 điểm gần nhất, mỗi phiếu nặng như nhau.

5. Chọn C và gamma

Kernel RBF có hai tham số cần chọn. C là mức phạt như ở mục 3. Gamma quyết định tầm ảnh hưởng của mỗi vectơ hỗ trợ: gamma nhỏ thì ảnh hưởng lan rộng và biên mượt, gamma lớn thì mỗi vectơ hỗ trợ chỉ ảnh hưởng quanh nó và biên gồ ghề.

```python
# -*- coding: utf-8 -*-
"""Bước 4: ảnh hưởng của gamma, rồi dò lưới C và gamma."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.svm import SVC

df = pd.read_csv("data/hai_vang_trang.csv")
X = df[["x1", "x2"]]
y = df["nhan"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

print("gamma   tap hoc   tap kiem tra   so vecto HT")
for g in [0.1, 1, 10, 100]:
    mh = SVC(kernel="rbf", gamma=g).fit(X_tr, y_tr)
    a_hoc = accuracy_score(y_tr, mh.predict(X_tr))
    a_kt = accuracy_score(y_te, mh.predict(X_te))
    print(f"  {g:5}  {a_hoc:.4f}    {a_kt:.4f}         "
          f"{len(mh.support_)}")
print()

# Dò lưới: thử mọi cặp bằng kiểm định chéo 5 phần, chỉ trên tập học
luoi = {"C": [0.1, 1, 10, 100], "gamma": [0.01, 0.1, 1, 10]}
tim = GridSearchCV(SVC(kernel="rbf"), luoi, cv=5)
tim.fit(X_tr, y_tr)

bang = pd.DataFrame(tim.cv_results_["params"])
bang["diem"] = tim.cv_results_["mean_test_score"]
print("Diem kiem dinh cheo, hang la C, cot la gamma")
print(bang.pivot(index="C", columns="gamma",
                 values="diem").round(4).to_string())
print()

print("Cap tot nhat:", tim.best_params_)
print(f"Diem kiem dinh cheo: {tim.best_score_:.4f}")
# GridSearchCV đã tự học lại với cặp tốt nhất trên toàn tập học
print(f"Accuracy tren tap kiem tra: {tim.score(X_te, y_te):.4f}")
```

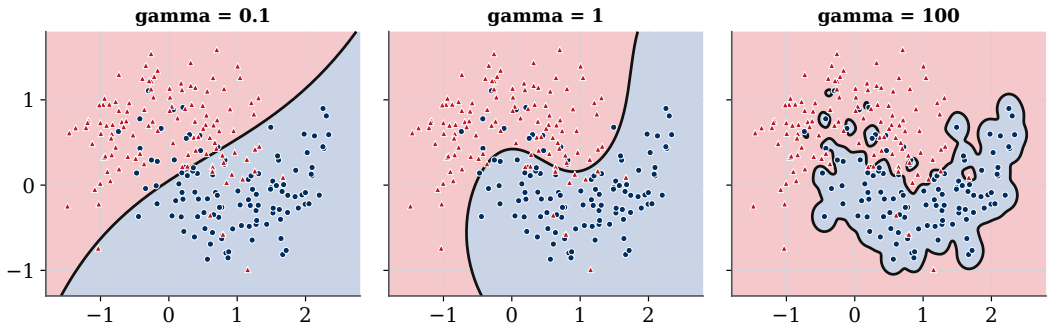
```
python code/g4_chon_tham_so.py
```

```
gamma   tap hoc   tap kiem tra   so vecto HT
    0.1  0.8356    0.8533         113
      1  0.8711    0.9200         88
     10  0.8978    0.9067         128
    100  0.9422    0.8133         217

Diem kiem dinh cheo, hang la C, cot la gamma
gamma   0.01    0.10    1.00    10.00
C
0.1    0.6578  0.8044  0.8444  0.8578
1.0    0.8044  0.8178  0.8667  0.8667
10.0   0.8178  0.8400  0.8711  0.8311
100.0  0.8178  0.8533  0.8711  0.7467

Cap tot nhat: {'C': 10, 'gamma': 1}
Diem kiem dinh cheo: 0.8711
Accuracy tren tap kiem tra: 0.9200
```

5.1. Ảnh hưởng của gamma



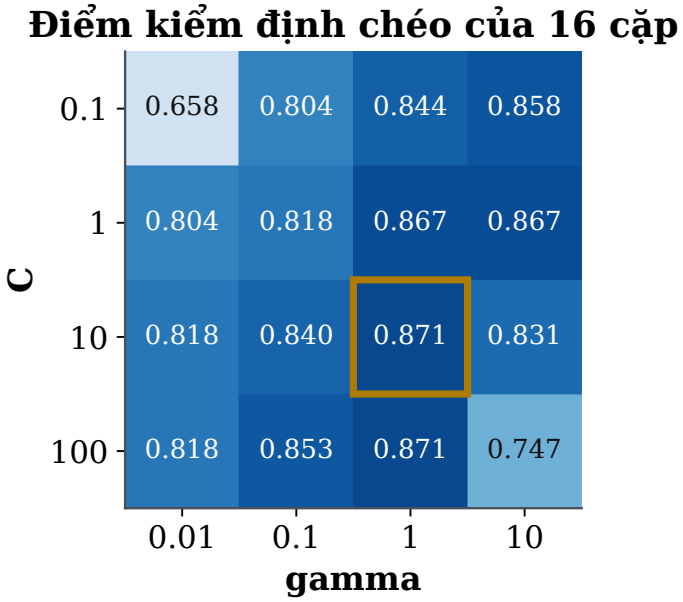
Hình 6: *Biên quyết định của SVM kernel RBF với ba giá trị gamma, cùng $C = 1$.*

Bảng đầu tiên và Hình 6 kể cùng một câu chuyện:

- Gamma bằng 0.1: biên gần như thẳng, tập kiểm tra chỉ 0.8533. Mô hình **cẩu thả**.
- Gamma bằng 1: biên cong theo đúng hình hai vầng trăng, tập kiểm tra 0.9200.
- Gamma bằng 100: biên thành những đốm nhỏ ôm lấy từng điểm, tập học lên 0.9422 nhưng tập kiểm tra tụt xuống 0.8133. Mô hình **học vẹt**, và 217 trên 225 điểm học trở thành vectơ hỗ trợ.

5.2. Dò lưới

Giá trị C tốt nhất còn phụ thuộc vào gamma, nên phải thử từng cặp. `GridSearchCV` thử 4 giá trị C nhân 4 giá trị gamma, mỗi cặp chấm bằng kiểm định chéo 5 phần trên tập học, tổng cộng 80 lần học.



Hình 7: *Điểm kiểm định chéo của 16 cặp tham số. Ô viền vàng là cặp được chọn.*

Cặp tốt nhất là $C = 10$ và gamma bằng 1, điểm kiểm định chéo 0.8711. Cặp $C = 100$ và gamma bằng 1 cũng được đúng 0.8711, và `GridSearchCV` chọn cặp đứng trước trong danh sách. Góc trái trên của bảng, cả hai tham số đều nhỏ, chỉ được 0.6578. Góc phải dưới, cả hai đều lớn, tụt xuống 0.7467 vì học vẹt.

GridSearchCV tự học lại

Dò xong, `GridSearchCV` tự học lại SVM với cặp tốt nhất trên toàn bộ tập học. Vì vậy `tim.score(X_te, y_te)` dùng được ngay, và cho accuracy 0.9200 trên tập kiểm tra.

Lưới thưa trước, lưới mịn sau

Nên bắt đầu với các giá trị cách nhau 10 lần như 0.1, 1, 10, 100. Tìm được vùng tốt rồi mới dò mịn quanh đó, vì mỗi giá trị thêm vào làm số lần học tăng lên nhiều.

6. Nhận dạng chữ số viết tay

Bộ dữ liệu `load_digits` gồm 1797 ảnh chữ số viết tay, mỗi ảnh 8×8 ô. Mỗi ô là một cột, giá trị là độ sáng từ 0 tới 16, nên mỗi ảnh là một dòng có 64 cột. Nhãn là chữ số từ 0 tới 9, tức bài toán có 10 lớp.

```python
# -*- coding: utf-8 -*-
"""Bước 5: nhận dạng chữ số viết tay bằng SVM."""

import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
print("Kich thuoc X:", X.shape)
print("Do sang nho nhat va lon nhat:", X.min(), X.max())
print("So anh moi chu so:", np.bincount(y))
print()

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

svm = SVC(kernel="rbf").fit(X_tr, y_tr)
du_doan = svm.predict(X_te)
print(f"SVM kernel RBF: accuracy {accuracy_score(y_te, du_doan):.4f}")
print("So vecto ho tro:", len(svm.support_), "tren", len(X_tr), "anh")
sai = np.flatnonzero(du_doan != y_te)
print("So anh doan sai:", len(sai), "tren", len(y_te))
for i in sai:
    print(f"  anh {i:3d}: that la {y_te[i]}, doan la {du_doan[i]}")
print()

# So với các mô hình đã học
mo_hinh = {
    "SVM co chuan hoa": make_pipeline(StandardScaler(), SVC()),
    "KNN, k = 5": KNeighborsClassifier(),
    "Logistic co chuan hoa": make_pipeline(
        StandardScaler(), LogisticRegression(max_iter=2000)),
}
print("Cac mo hinh khac, accuracy tren tap kiem tra")
for ten, mh in mo_hinh.items():
    mh.fit(X_tr, y_tr)
    acc = accuracy_score(y_te, mh.predict(X_te))
    print(f"  {ten:22s} {acc:.4f}")
```

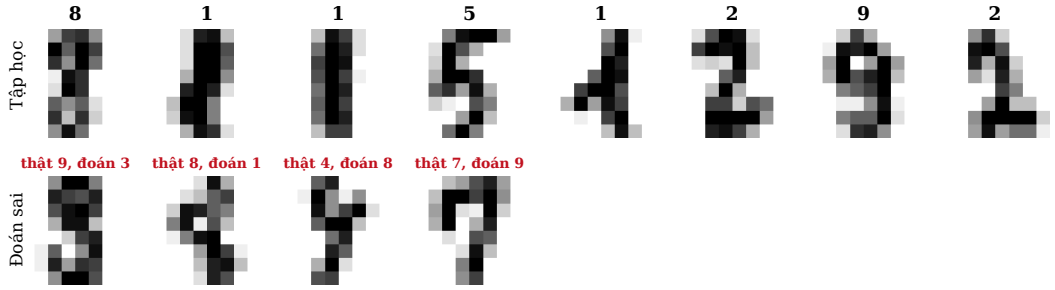
```
python code/g5_chu_so.py
```

```
Kich thuoc X: (1797, 64)
Do sang nho nhat va lon nhat: 0.0 16.0
So anh moi chu so: [178 182 177 183 181 182 181 179 174 180]

SVM kernel RBF: accuracy 0.9911
So vecto ho tro: 626 tren 1347 anh
So anh doan sai: 4 tren 450
  anh   6: that la 9, doan la 3
  anh  18: that la 8, doan la 1
  anh 161: that la 4, doan la 8
```

```
  anh 428: that la 7, doan la 9

Cac mo hinh khac, accuracy tren tap kiem tra
  SVM co chuan hoa       0.9778
  KNN, k = 5             0.9867
  Logistic co chuan hoa  0.9711
```



Hình 8: *Hàng trên là tám ảnh đầu tiên của tập học kèm nhãn. Hàng dưới là bốn ảnh SVM đoán sai trên tập kiểm tra.*

6.1. Đọc kết quả

Mười chữ số có số ảnh gần bằng nhau, từ 174 tới 183 ảnh, nên accuracy là thước đo hợp lý ở đây.

SVM kernel RBF để mặc định đạt 0.9911, chỉ đoán sai 4 trên 450 ảnh kiểm tra. Nhìn hàng dưới của Hình 8, những ảnh này viết khá khó đọc, ví dụ một chữ 9 bị đoán thành chữ 3. SVM giữ 626 vectơ hỗ trợ trên 1347 ảnh học.

Nhiều hơn hai lớp

SVM gốc chỉ tách được hai lớp. Với 10 lớp, `SVC` tự học một SVM cho mỗi cặp chữ số, tổng cộng 45 cặp. Ảnh mới được cả 45 SVM bỏ phiếu, chữ số nhiều phiếu nhất thắng. Ta không phải viết thêm dòng mã nào.

6.2. So với các mô hình khác

KNN đạt 0.9867 và hồi quy logistic có chuẩn hoá đạt 0.9711. Điều đáng chú ý là SVM **có chuẩn hoá** lại kém hơn, chỉ 0.9778.

Không phải lúc nào cũng chuẩn hoá

Ở đây mọi cột đã cùng thang đo, độ sáng từ 0 tới 16. Các ô ở mép ảnh gần như lúc nào cũng trắng, độ lệch chuẩn rất nhỏ. Chuẩn hoá chia cho độ lệch chuẩn nhỏ đó nên phóng to những ô gần như vô nghĩa này lên ngang các ô ở giữa ảnh. Luôn nhìn dữ liệu trước khi quyết định chuẩn hoá.

7. Bài tập

Sinh viên tạo thư mục `baitap06` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap06/bai1.py`, và trong mã vẫn viết `data/...`. Bài nộp gồm tệp

mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Tính tay khoảng cách tới ranh giới

Dùng w_1, w_2, b in ra ở mục 2, tự tính $f(\boldsymbol{x})$ cho hai điểm $(3, 3)$ và $(5, 4.5)$ bằng phép nhân và cộng, không dùng `decision_function`.
Khoảng cách từ một điểm tới đường ranh giới bằng $|f(\boldsymbol{x})|/\sqrt{w_1^2 + w_2^2}$. Tính khoảng cách của hai điểm trên, và của ba vectơ hỗ trợ. Vì sao khoảng cách của ba vectơ hỗ trợ bằng đúng một nửa độ rộng lề?

Bài tập 2: Thêm một điểm

Thêm vào 24 điểm của mục 2 một điểm lớp 1 tại $(-1, -1)$, học lại và in w_1, w_2, b. Sau đó bỏ điểm đó, thay bằng một điểm lớp 1 tại $(4.5, 4)$, học lại và in lần nữa.
Lần nào đường ranh giới thay đổi? Giải thích bằng khái niệm vectơ hỗ trợ.

Bài tập 3: Kernel đa thức

Trên bộ hai vầng trăng, thử `SVC(kernel="poly", degree=d)` với d bằng 2, 3 và 4, có chuẩn hoá bằng `make_pipeline`. Với mỗi bậc, dò C trong 0.1, 1, 10, 100 bằng `GridSearchCV`.
Bậc nào tốt nhất? Có vượt được kernel RBF ở mục 5 không? Gợi ý, trong lưới tham số của pipeline, tên tham số viết là `"svc__C"`.

Bài tập 4: Dò lưới mịn

Quanh cặp tốt nhất của mục 5, dò lại với C trong 3, 10, 30 và gamma trong 0.3, 1, 3. Điểm kiểm định chéo tốt nhất có cao hơn 0.8711 không? Accuracy trên tập kiểm tra có đổi không? Nhận xét về cái lợi của việc dò thật mịn.

Bài tập 5: Chữ số viết tay, những lỗi nào hay gặp

Dùng `confusion_matrix` trong `sklearn.metrics` để in bảng nhầm lẫn 10 hàng 10 cột của SVM ở mục 6. Hàng là chữ số thật, cột là chữ số đoán.
Đọc bảng và cho biết những cặp chữ số nào bị nhầm. Dùng `plt.imshow(X_te[i].reshape(8, 8), cmap="gray_r")` để vẽ và lưu từng ảnh bị đoán sai, rồi nhận xét vì sao SVM nhầm. Chương 7 sẽ học kỹ về bảng nhầm lẫn.

Bài tập 6: SVM trên dữ liệu lớn

Tạo dữ liệu bằng `make_classification(n_samples=n, n_features=20, random_state=0)` với n bằng 2000, 5000, 10000 và 20000. Với mỗi cỡ, đo thời gian `.fit` của `SVC()` và của `LinearSVC()` bằng mô đun `time`.
Lập bảng rồi nhận xét: khi dữ liệu tăng gấp đôi, thời gian học của mỗi mô hình tăng khoảng bao nhiêu lần? Kết quả thời gian khác nhau giữa các máy là chuyện bình thường, chỉ cần so tương đối.

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 1 và bài 2 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có hiểu lề và vectơ hỗ trợ hay không.

8. Phụ lục A, bảng tra nhanh

Bảng 2: *Các hàm mới xuất hiện trong bài 6.*

Lệnh	Thư viện	Công dụng
`SVC`	scikit-learn	SVM cho bài toán phân loại, mặc định kernel RBF.
`kernel`	scikit-learn	Loại kernel: `"linear"`, `"poly"`, `"rbf"`.
`C`	scikit-learn	Mức phạt cho điểm vi phạm lề, mặc định bằng 1.
`gamma`	scikit-learn	Tầm ảnh hưởng của mỗi vectơ hỗ trợ, mặc định `"scale"`.
`.coef_`, `.intercept_`	scikit-learn	Hệ số w và b, chỉ có với kernel tuyến tính.
`.support_`	scikit-learn	Chỉ số của các vectơ hỗ trợ trong tập học.
`.decision_function`	scikit-learn	Giá trị $f(\boldsymbol{x})$ của từng điểm.
`GridSearchCV`	scikit-learn	Dò lưới tham số bằng kiểm định chéo.
`.best_params_`	scikit-learn	Cặp tham số tốt nhất sau khi dò lưới.
`make_blobs`	scikit-learn	Sinh các cụm điểm ngẫu nhiên để thử nghiệm.
`load_digits`	scikit-learn	Nạp bộ 1797 ảnh chữ số viết tay.

Bảng 3: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
24 điểm tách được, độ rộng lề	2.21
24 điểm tách được, số vectơ hỗ trợ	3
Sinh viên, số vectơ hỗ trợ khi $C = 0.01$ và $C = 100$	74 và 27
Ba cột, không chuẩn hoá và có chuẩn hoá	0.6000 và 0.9000
Hai vầng trăng, kernel tuyến tính và RBF	0.8667 và 0.9200
Gamma bằng 100, tập học và tập kiểm tra	0.9422 và 0.8133
Dò lưới, cặp tốt nhất	$C = 10$, gamma bằng 1
Dò lưới, kiểm định chéo và tập kiểm tra	0.8711 và 0.9200
Chữ số, SVM kernel RBF	0.9911, sai 4 trên 450
Chữ số, KNN và logistic	0.9867 và 0.9711

9. Phụ lục B, lỗi thường gặp

Bảng 4: *Các lỗi hay gặp riêng của bài SVM.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
Accuracy đúng bằng mức đoán bừa	Quên chuẩn hoá trong khi các cột khác thang đo. Dùng `make_pipeline(StandardScaler(), SVC())`.
`AttributeError: coef_ is only available when using a linear kernel`	Chỉ kernel tuyến tính mới có w và b. Với kernel RBF, hãy xem `.support_` và `.dual_coef_`.
`Invalid parameter 'C' for estimator Pipeline`	Dò lưới trên pipeline phải ghi tên bước phía trước. Viết `"svc__C"` thay cho `"C"`.
Học rất lâu không xong	Dữ liệu quá lớn cho SVM kernel. Thử trên một mẫu nhỏ trước, hoặc dùng `LinearSVC`.
Accuracy tập học rất cao, tập kiểm tra thấp	Gamma hoặc C quá lớn. Dò lại bằng `GridSearchCV`.
Gần như mọi điểm là vectơ hỗ trợ	C quá nhỏ hoặc gamma quá lớn.
Kết quả khác tài liệu	Quên `random_state=17` hoặc `stratify=y` khi chia dữ liệu.

Một lời khuyên chung

Với SVM, câu hỏi đầu tiên vẫn là: các cột đã cùng thang đo chưa? Câu hỏi thứ hai là: đã dò C và gamma bằng kiểm định chéo chưa? Trả lời xong hai câu này thì phần lớn lỗi của SVM đã được loại bỏ.